PathGuard

On-Device Spatial Safety Intelligence

Real-time construction site hazard detection + on-device vision-language scene understanding

Component	What It Does	Key Technology
PathGuard HUD	Real-time corridor-based obstacle detection with persistence state machine	Classical CV, GroundedDINO, Depth Anything V2, SAM2
Cactus Narrator	On-device VLM video narration with hybrid cloud routing	LFM2.5-VL-1.6B (Apple ANE), Gemini 2.5 Flash Lite
Integration Layer	Dynamic DINO prompts + enriched RCP telemetry	File-based + session-state coupling

UMD x Ironsite Startup Shell Hackathon 2026

February 2026

Every layer degrades gracefully. The system is useful even when every ML model is unavailable.

Table of Contents

1. Executive Summary 3
2. Problem Framing 3
3. Process -- How We Got Here 3
4. Technical Approach (From First Principles) 4
4.1 Corridor-First Spatial Reasoning 4
4.2 Classical CV as the Always-On Safety Net 4
4.3 Zero-Shot Detection via GroundedDINO 4
4.4 Monocular Depth for Relative Urgency 4
4.5 Persistence-Based State Machine 5
4.6 On-Device VLM with Hybrid Cloud Routing 5
4.7 Dynamic Prompt Generation and RCP Telemetry 5
4.8 Mask Fusion and HUD Rendering 5
5. System Pipeline 5
PathGuard HUD Pipeline (per frame) 6
Cactus Narrator Pipeline (per sampled frame) 6
6. Architecture Overview 7
7. Hazard State Machine 7
8. What the Demo Shows 8
9. Repository Layout 9
10. Installation and Run 9
PathGuard HUD (any platform with Python 3.11+) 9
Cactus Narrator (macOS Apple Silicon only) 9
API Key 10
Run Commands 10
11. Key Defaults We Tuned 10
12. How the Two Parts Connect 10
What requires manual steps 11
13. Findings 11
14. Known Limitations 11
15. Model References 11
16. Future Extensions 12
Appendix A: Key Code Excerpts 12
A.1 Classical CV Fallback (pathguard/fallback.py) 12
A.2 Event State Machine (pathguard/events.py) 13
A.3 Hybrid Cloud Routing (narrator/cactus_vl.py, lines 162-226) 14
A.4 Dynamic Prompt Merging (integration/dynamic_prompts.py) 16
Closing Note 17

1. Executive Summary

PathGuard is a two-part spatial safety system for construction workers. Both parts run as pages inside one Streamlit multipage app and communicate through shared files and session state.

Part 1 -- PathGuard HUD: A real-time corridor-based obstacle detection pipeline. It defines a trapezoidal walking-lane ROI, runs classical CV every frame, and optionally schedules GroundedDINO (zero-shot detection), SAM2 (segmentation refinement), and Depth Anything V2 (relative distance). A persistence-based state machine governs transitions between CLEAR, OBSTACLE AHEAD, and TRIP RISK.

Part 2 -- Cactus Narrator: An on-device VLM video narrator. It runs LFM2.5-VL-1.6B on the Apple Neural Engine with Gemini cloud fallback, producing a scene transcript, dynamic GroundedDINO prompts, and structured RCP site telemetry.

How they relate: Cactus generates scene-specific detection prompts that replace PathGuard's static defaults. PathGuard's spatial event data (occupancy, depth, trip risk) feeds back into the RCP telemetry to spatially ground what would otherwise be purely semantic observations.

Design philosophy: Every layer degrades gracefully. The classical CV fallback runs on any CPU with no ML models loaded. GroundedDINO, Depth Anything V2, SAM2, the local VLM, and the Gemini cloud API are all optional escalations. The system is useful even when every ML model is unavailable.

2. Problem Framing

Generic object detection on construction bodycam footage is insufficient for one specific question: "Is there a hazard in my walking path right now?"

- Detectors flag objects everywhere in the frame, not just in the walking path.
- Static prompt lists miss site-specific clutter that changes between sites and shifts.
- GPU-dependent pipelines cannot deploy to phones, Raspberry Pis, or hardhats.
- Single-model systems go dark when that model is slow, unavailable, or wrong.

Our approach layers multiple techniques so that each one adds intelligence without being required. The cheapest layer runs every frame. Heavier models run on schedule. The VLM contributes asynchronously via the filesystem.

3. Process -- How We Got Here

We started with a single GroundedDINO detector running on every frame. It worked on clean, well-lit footage -- and fell apart on real bodycam video. Shaky camera motion produced blurred frames that generated phantom detections. Objects outside the walking path triggered constant false alarms. Inference latency meant the safety layer went silent for hundreds of milliseconds between frames.

First pivot: corridor-first reasoning. Instead of detecting globally and filtering, we defined the walking lane first. This single decision eliminated the majority of irrelevant detections and let us ask a spatially specific question.

Second pivot: classical CV as the backbone. We wrote a Canny + morphology fallback that runs on every frame with zero ML overhead. This gave us an always-on safety floor that persists even when GroundedDINO is loading, crashing, or too slow.

Third pivot: blur quality gate. Motion blur was generating dense false edges inside the corridor. Adding a Laplacian variance check to suppress low-quality frames cut false TRIP RISK alarms substantially.

Fourth pivot: persistence-based state machine. Threshold-only hazard classification flickered between states on every frame. Requiring multiple consecutive hazard frames before alarming -- but clearing instantly -- gave stable, usable behavior.

Fifth pivot: hybrid cloud routing. Running the VLM locally on Apple Neural Engine was fast but occasionally uncertain. Rather than accepting low-confidence outputs, we added a confidence gate that routes to Gemini only when needed, with a cooldown timer to cap API costs.

Sixth pivot: dynamic prompts. The static prompt list we hand-tuned for general construction sites kept missing site-specific objects. Having the VLM observe the scene first and generate a tailored noun list for GroundedDINO closed the vocabulary gap.

Each layer was added because the previous configuration failed on a specific class of input. Nothing was designed in the abstract -- every component exists because we watched the system fail without it.

4. Technical Approach (From First Principles)

4.1 Corridor-First Spatial Reasoning

Rather than detecting objects globally and filtering later, we define the region of interest first. A trapezoidal corridor polygon approximates the worker's walking lane using four fractional parameters (bottom_width_frac, top_width_frac, height_frac, center_x_frac). Every subsequent computation -- fallback edges, detection filtering, occupancy scoring, depth estimation -- operates relative to this corridor mask.

This inverts the typical pipeline: instead of "detect everything, then decide what matters," we ask "what's in the path?" first.

4.2 Classical CV as the Always-On Safety Net

On every frame, fallback.py runs a lightweight pipeline:

1. Grayscale conversion and Gaussian blur
2. Laplacian variance as a blur quality gate -- if the frame is too blurry, we suppress false positives by marking it low_quality
3. Canny edge detection, masked to the corridor
4. Morphological close + dilate to merge fragmented edges into connected components
5. Each component above min_component_area pixels contributes to the obstacle mask
6. Trip risk heuristic: components with area >= trip_component_area and aspect ratio >= 6.0 (elongated shapes like hoses, cables, rebar) set a trip_risk_flag
7. Occupancy score = obstacle_area / corridor_area

This runs at full frame rate on any CPU. No models, no weights, no GPU.

4.3 Zero-Shot Detection via GroundedDINO

detect.py wraps GroundedDINO with a dual-backend fallback strategy: it first tries the groundeddino_vl PyPI package, and if that fails, falls back to Hugging Face's AutoModelForZeroShotObjectDetection using IDEA-Research/grounding-dino-tiny. Both backends are loaded once and reused.

Detections are filtered to the corridor by checking what fraction of each bounding box overlaps the corridor mask (min_overlap=0.10). Only in-corridor detections contribute to the obstacle mask.

GroundedDINO runs on a schedule (dino_interval frames) or opportunistically when the fallback detector's occupancy score approaches the threshold or a trip risk flag is already set.

4.4 Monocular Depth for Relative Urgency

depth.py wraps Depth Anything V2 to produce a closeness map (not a metric depth map). The raw depth output

undergoes min-max normalization, auto-polarity detection (compares median depth of the top vs. bottom fifth of the frame), and bucketing into NEAR (>= 0.66), MID (>= 0.33), FAR (< 0.33).

For each detected object, box_closeness computes the median closeness within the bounding box. This provides qualitative urgency cues without claiming metric accuracy.

4.5 Persistence-Based State Machine

events.py implements a state machine that resists both false positives and false negatives:

- Slow to alarm: Occupancy must exceed occ_thresh for persistence_frames consecutive frames before transitioning from CLEAR to OBSTACLE AHEAD (or TRIP RISK if the trip flag is also set).
- Fast to clear: A single frame below threshold resets the counter and returns to CLEAR.
- Debounce: Duplicate state-transition events within a 2-second window are suppressed.
- Bounded memory: The event log is capped at 200 entries (circular buffer behavior).

4.6 On-Device VLM with Hybrid Cloud Routing

cactus_vl.py implements a confidence-gated hybrid routing strategy:

1. The local LFM2.5-VL-1.6B model (via the Cactus C++ engine on Apple Neural Engine) processes the frame.
2. If the local model's confidence >= confidence_threshold, the local response is used directly.
3. If confidence is below threshold and a cooldown_seconds timer has elapsed, the frame is Base64-encoded and POSTed to Vertex AI's gemini-2.5-flash-lite streaming endpoint.
4. If still on cooldown, the local result is returned with a warning annotation.

This keeps cloud costs bounded while providing a quality floor.

4.7 Dynamic Prompt Generation and RCP Telemetry

After processing a video (or manually triggering on a webcam session), two Gemini calls run in parallel threads:

- generate_dino_prompt (via gemini-2.5-flash-lite): extracts a dot-separated noun list from the full transcript.
- generate_rcp_telemetry (via gemini-2.5-pro): extracts structured JSON following a 7-category RCP schema (activity, equipment, materials, tools, workforce, safety/PPE, hazards). Output is hardened with json_repair.

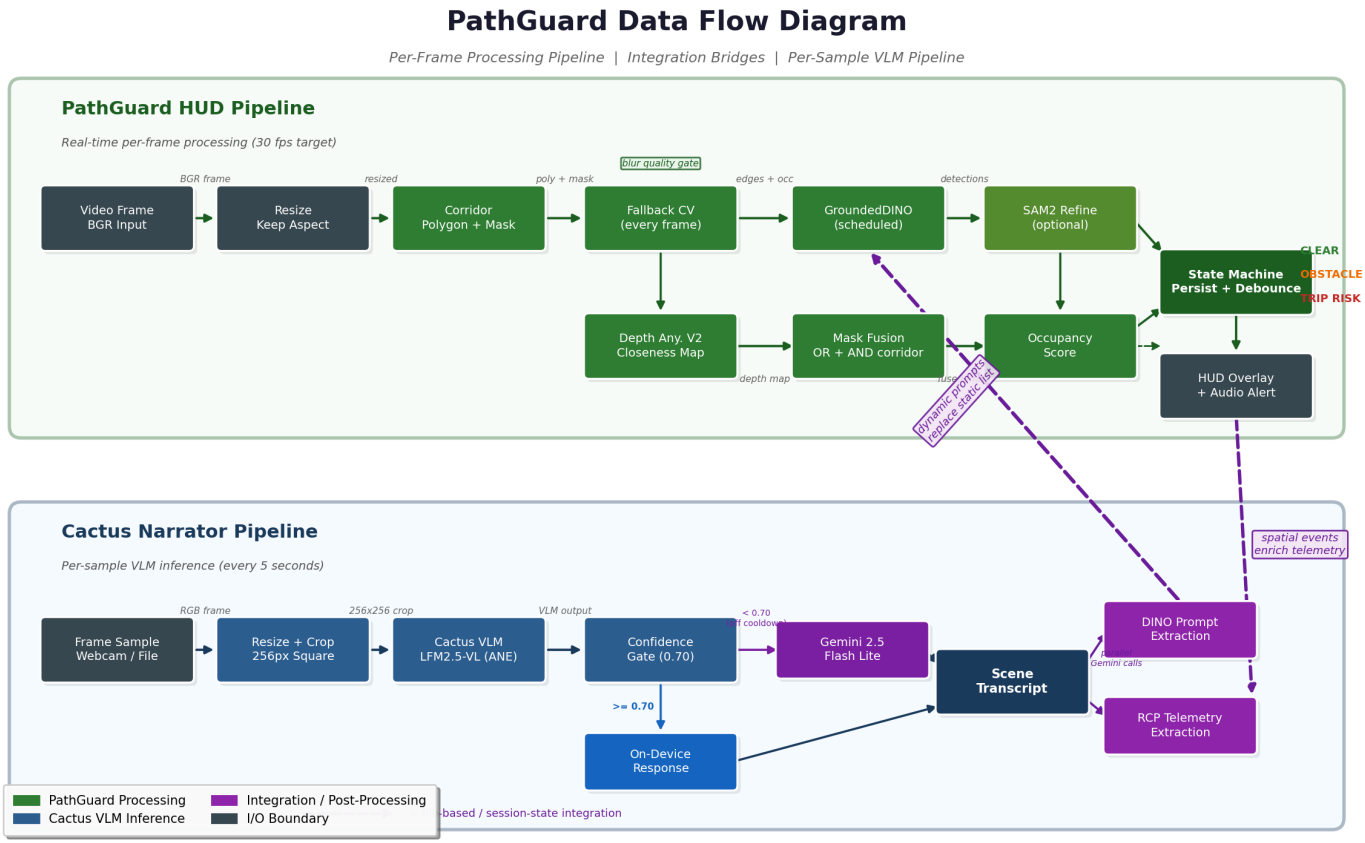
4.8 Mask Fusion and HUD Rendering

Each frame's final obstacle mask is the bitwise OR of the fallback CV mask and the GroundedDINO detection mask (intersected with the corridor). The HUD overlay renders:

- Corridor polygon outline (yellow)
- Obstacle mask (semi-transparent red)
- Detection bounding boxes with labels, confidence scores, and depth tags
- Status chip (green/orange/red) showing the current state

5. System Pipeline

The diagram below shows the high-level data flow through both pipelines and their integration point.



PathGuard HUD Pipeline (per frame)

1. Read frame from video file via OpenCV VideoCapture
2. Resize to output_width preserving aspect ratio
3. Compute corridor polygon and binary mask from CorridorParams
4. Run fallback CV: blur gate, Canny edges, morphology, connected components, occupancy score, trip risk flag
5. If GroundedDINO enabled and scheduled (or occupancy rising): run zero-shot detection, filter to corridor
6. If Depth Anything V2 enabled and scheduled (or detections exist): estimate closeness map, tag NEAR/MID/FAR
7. If SAM2 enabled: refine detection boxes into segmentation masks; otherwise rasterize boxes
8. Fuse fallback mask with detection mask
9. Compute final occupancy score
10. Update state machine with occupancy, trip flag, and quality gate
11. Render HUD overlay, yield frame + metrics to Streamlit
12. If simulate_realtime is on, sleep to match source video FPS
13. On video completion: look for matching Cactus RCP file, merge spatial data, offer enriched telemetry download

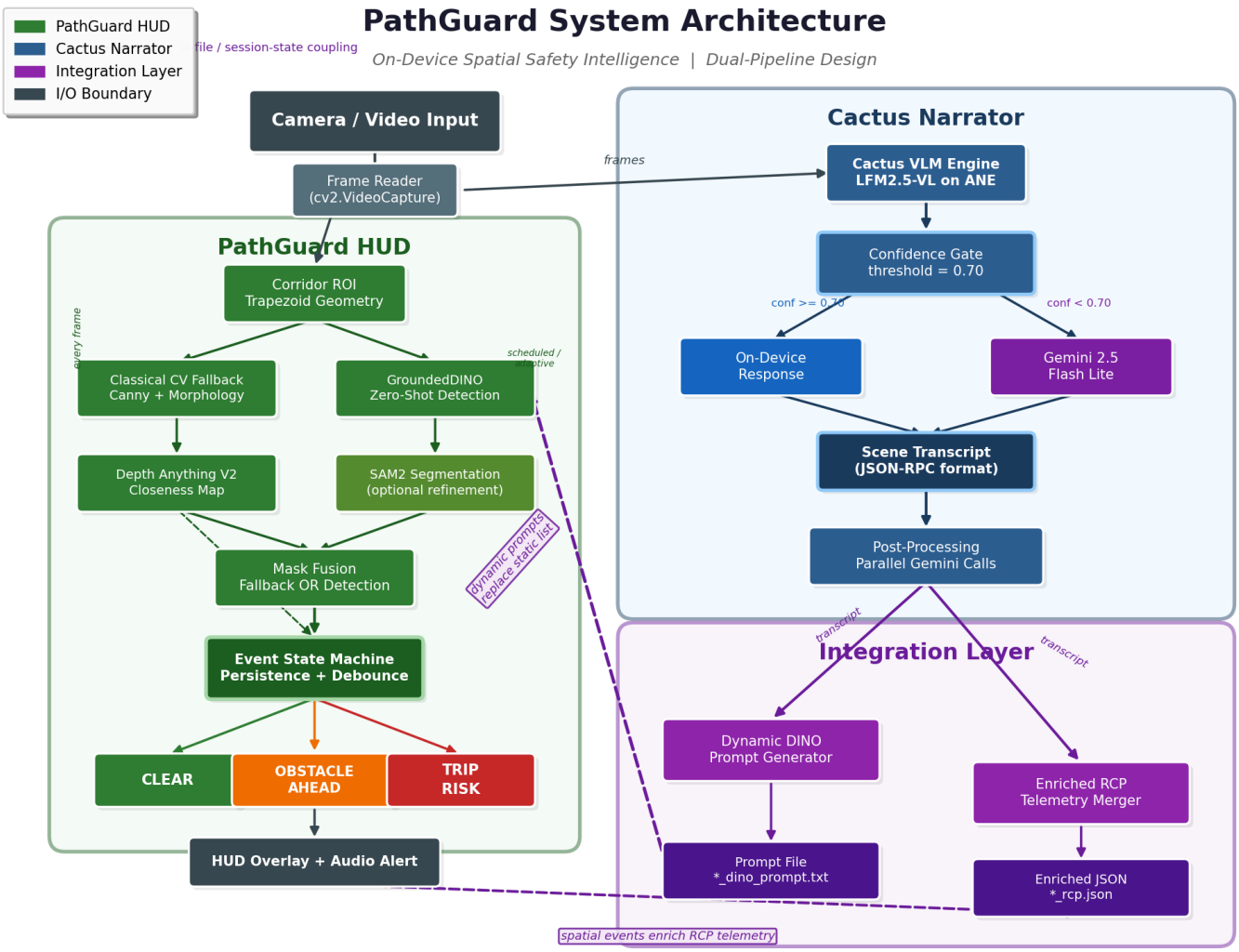
Cactus Narrator Pipeline (per sampled frame)

1. Accept input: live webcam (via streamlit-webrtc) or uploaded video file
2. Sample frames at configurable interval (sample_every seconds)
3. Resize and center-crop to square for NPU fixed-shape constraint
4. Run local VLM inference via Cactus C++ engine
5. Evaluate confidence; route to Gemini if below threshold and off cooldown

6. Append result to transcript (JSON-RPC format with timestamp, text, source, confidence)
7. Save transcript continuously to transcripts/ directory
8. On completion: run parallel generate_dino_prompt + generate_rcp_telemetry via Gemini
9. Save DINO prompt file and RCP JSON to transcripts/; sync to session state for cross-page access

6. Architecture Overview

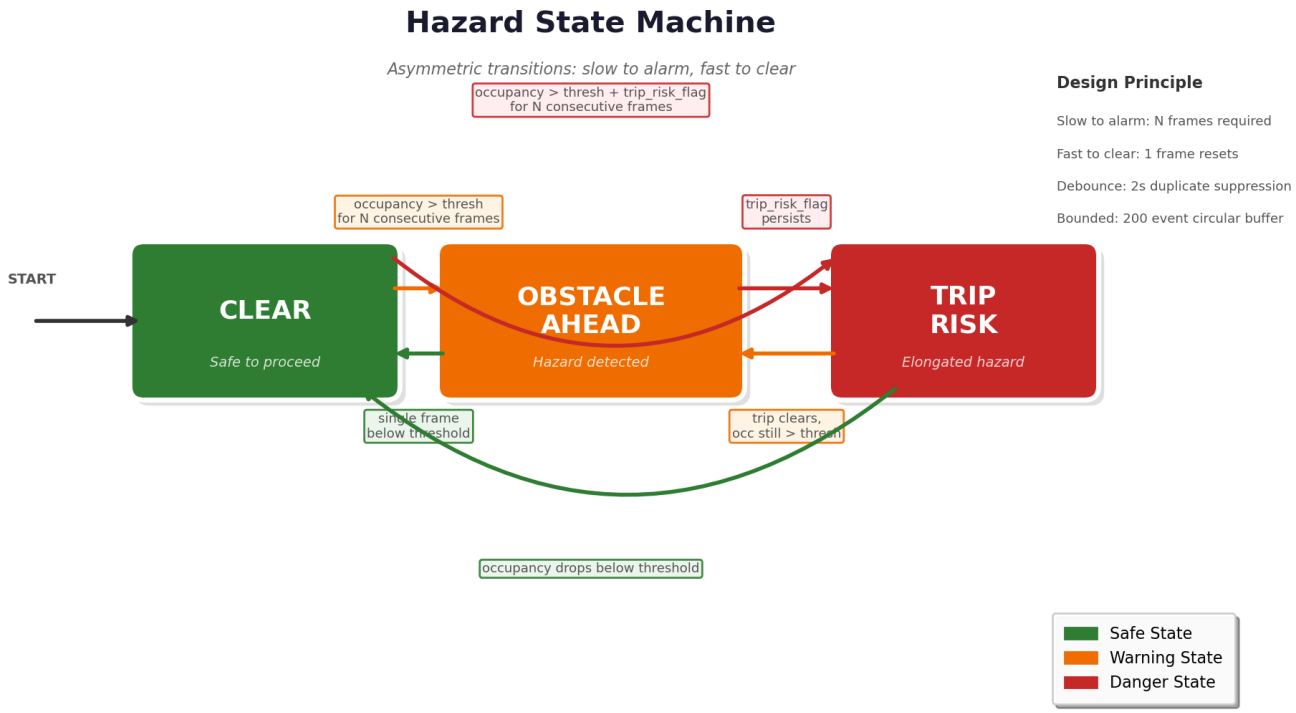
The system has three subsystems connected by file-based and session-state coupling (not a live data stream). The full architecture is shown below.



> *Green = PathGuard spatial reasoning. Blue = Cactus Narrator inference. Purple = Integration bridges. Gray = I/O boundaries. Dashed arrows = file-based or session-state coupling (not a live data stream).*

7. Hazard State Machine

The state machine is deliberately asymmetric: slow to alarm (requires persistence_frames consecutive hazard frames), fast to clear (single clean frame resets). This trades a few hundred milliseconds of detection latency for dramatically fewer false alarms.



Transition details:

From	To	Condition
CLEAR	OBSTACLE AHEAD	occupancy > occ_thresh for persistence_frames frames
CLEAR	TRIP RISK	occupancy > occ_thresh + trip_risk_flag for persistence_frames frames
OBSTACLE AHEAD	CLEAR	Single frame below threshold
OBSTACLE AHEAD	TRIP RISK	trip_risk_flag persists
TRIP RISK	OBSTACLE AHEAD	trip_risk_flag clears, occupancy still high
TRIP RISK	CLEAR	Occupancy drops below threshold

8. What the Demo Shows

- Live video playback with a yellow corridor overlay, red obstacle mask, and green detection boxes
- Real-time status chip cycling between CLEAR (green), OBSTACLE AHEAD (orange), and TRIP RISK (red)
- FPS counter, corridor occupancy score, relative distance bucket, and depth model status
- Event log table accumulating timestamped state transitions
- Audio alarm (two-tone beep) on TRIP RISK transitions
- Sidebar controls for corridor geometry, detection intervals, thresholds, and model toggles
- Dynamic prompt source selector showing available Cactus-generated prompt files
- Cactus Narrator page with live webcam or video file input, side-by-side frame preview and narration
- On-device vs. cloud source indicators on each narration entry
- Inference metrics (latency, tokens/sec, RAM, confidence, cloud handoff flag)
- Post-processing output: GroundedDINO noun list and RCP telemetry JSON
- Enriched telemetry download button merging PathGuard spatial data into Cactus RCP

9. Repository Layout

File	Description
Home.py	Streamlit multipage entrypoint and landing page
pages/1_PathGuard_HUD.py	PathGuard spatial safety HUD page
pages/2_Cactus_Narrator.py	Cactus on-device video narrator page
pathguard/__init__.py	Package marker
pathguard/config.py	Corridor/runtime params, static prompt list, video discovery
pathguard/realtime.py	Main frame loop: scheduling, fusion, overlay rendering
pathguard/corridor.py	Corridor polygon geometry, binary mask, intersection scoring
pathguard/fallback.py	Classical CV obstacle detector: Canny + morphology + trip heuristic
pathguard/events.py	Persistence/debounce state machine (CLEAR/OBSTACLE/TRIP)
pathguard/detect.py	GroundedDINO wrapper with dual-backend fallback (PyPI + HF)
pathguard/depth.py	Depth Anything V2 wrapper, closeness map, NEAR/MID/FAR buckets
pathguard/segment.py	Optional SAM2 segmentation refinement, box rasterization fallback
pathguard/audio_alerts.py	Trip Risk audio alarm (winsound on Windows, terminal bell elsewhere)
narrator/__init__.py	Package marker
narrator/cactus_vl.py	Cactus engine wrapper, Gemini hybrid routing, DINO/RCP generation
integration/__init__.py	Package marker
integration/dynamic_prompts.py	Parses Cactus DINO prompt files, merges with static list
integration/enriched_telemetry.py	Injects PathGuard spatial events into RCP telemetry JSON
scripts/download_models.sh	Downloads + converts LFM2.5-VL weights to INT8
smoke_test.py	Import verification for all packages
run_combined.sh	Launches the full multipage Streamlit app
run_pathguard.sh	Launches PathGuard HUD page standalone
run_narrator.sh	Launches Cactus Narrator page standalone
setup.sh	One-shot bootstrap: venv, deps, Cactus C++ build, weight download
requirements_pathguard.txt	Dependencies for PathGuard HUD
requirements_narrator.txt	Dependencies for Cactus Narrator
.streamlit/config.toml	Streamlit config (maxUploadSize = 1000 MB)
.gitignore	Excludes weights, videos, transcripts, build artifacts, secrets

> *The cactus/ directory (C++ engine source + Python bindings) is not included in this repository. It must be cloned separately and compiled via setup.sh. The weights/, transcripts/, and video/ directories are gitignored and created at runtime.*

10. Installation and Run

PathGuard HUD (any platform with Python 3.11+)

```
python -m venv .venv
source .venv/bin/activate          # Linux/macOS
# .venv\Scripts\activate           # Windows

pip install torch torchvision --index-url https://download.pytorch.org/whl/cu126
pip install -r requirements_pathguard.txt
```

> *GroundedDINO and Depth Anything V2 download weights automatically from Hugging Face on first run. If no CUDA GPU is available, both fall back to CPU.*

Cactus Narrator (macOS Apple Silicon only)

```
chmod +x setup.sh && ./setup.sh
pip install -r requirements_narrator.txt
pip install -r requirements_pathguard.txt
```

API Key

.env file in project root

```
GEMINI_API_KEY="your_vertex_ai_key_here"
```

> *The API key is used by Cactus Narrator's cloud fallback and telemetry generation. PathGuard's spatial pipeline does not require it.*

Run Commands

```
./run_combined.sh     # Full multipage app
./run_pathguard.sh    # PathGuard HUD only
./run_narrator.sh     # Cactus Narrator only

# Windows:
set PYTHONPATH=.;cactus/python/src
streamlit run Home.py
```

The app opens at http://localhost:8501.

11. Key Defaults We Tuned

Parameter	Default	Rationale
bottom_width_frac	0.90	Corridor spans most of frame bottom (walking surface)
top_width_frac	0.34	Perspective narrowing at far end of corridor
height_frac	0.64	Lower two-thirds; upper third is sky/walls
occ_thresh	0.015	1.5% corridor occupancy triggers attention
persistence_frames	2	Two consecutive hazard frames before alarm
blur_thresh	20.0	Laplacian variance below 20 = low quality frame
trip_component_area	180 px	Minimum area to consider for trip risk
trip_aspect_ratio	6.0	Elongated shapes trigger trip flag
dino_interval	10	Run GroundedDINO every 10th frame
depth_interval	10	Run depth estimation every 10th frame
confidence_threshold	0.70	VLM confidence below 70% triggers Gemini fallback
cooldown_seconds	10	Min gap between Gemini API calls
max_prompts	50	Cap on merged prompt list (tokenizer overflow prevention)
debounce_sec	2.0	Suppress duplicate events within 2s window
sample_every	5.0s	Narrator samples one frame every 5 seconds

12. How the Two Parts Connect

The integration between PathGuard HUD and Cactus Narrator is real but manually sequenced. They do not run simultaneously on the same video in a shared pipeline.

Cactus to PathGuard (dynamic prompts): When Cactus finishes processing a video, it saves a *_dino_prompt.txt file to transcripts/ and syncs the prompt text to st.session_state. The PathGuard HUD page scans transcripts/ for available prompt files. The user selects a source from a sidebar dropdown. dynamic_prompts.py parses the dot-separated noun list and merges it with the static defaults (dynamic prompts first, capped at 50 total).

PathGuard to Cactus (enriched telemetry): When PathGuard finishes processing a video, it looks for a matching

*_rcp.json file in transcripts/. If found, enriched_telemetry.py deep-copies the RCP JSON and injects a pathguard_spatial section containing total hazard events, trip risk count, obstacle count, peak occupancy, and a timestamped event log. It also cross-references timestamps within a 5-second window.

What requires manual steps

- The user must run Cactus on a video first, then switch to PathGuard and select the generated prompt file.
- For the enriched telemetry merge, both systems must have processed the same video and the RCP file must exist in transcripts/.

13. Findings

Corridor-centric fallback detection is surprisingly effective. Restricting attention to the walking lane eliminates most irrelevant detections. The classical CV pipeline alone catches the majority of in-path obstacles without any ML models.

The blur quality gate matters more than expected. Without it, camera shake and motion blur generate dense phantom edges that spike the occupancy score and cause false TRIP RISK alarms.

Zero-shot labels are valuable for explainability, not primary detection. GroundedDINO adds human-readable labels and catches objects the edge detector misses, but its latency makes it unsuitable as the sole detection layer.

Relative depth is useful as a qualitative cue, not a measurement. NEAR/MID/FAR buckets give the worker a sense of urgency without implying metric accuracy.

Dynamic prompts outperform static prompts. Scene-specific nouns extracted by Cactus consistently surface objects the static default list misses.

Hybrid on-device/cloud routing is practical. The local VLM handles the majority of frames, and Gemini fills in only when confidence drops. The cooldown timer keeps API costs predictable.

The state machine's asymmetry is the key to demo quality. Slow-to-alarm, fast-to-clear eliminates the flickering behavior that makes threshold-only systems unusable.

json_repair is essential for LLM-generated JSON. Gemini occasionally produces trailing commas, unescaped quotes, or truncated output. Hardening the parser prevents silent failures.

14. Known Limitations

- Relative distance is not metric distance. NEAR/MID/FAR are qualitative.
- Static corridor geometry. The trapezoid does not adapt to camera rotation or curved paths.
- Cactus engine requires macOS Apple Silicon. PathGuard HUD works on any platform.
- No temporal tracking. Each frame is processed independently.
- Cloud fallback depends on Gemini API availability and rate limits.
- GroundedDINO prompt coverage is bounded by what Cactus observes.
- The integration is file-based and manually sequenced. No live streaming bridge.
- SAM2 segmentation is stubbed. The wrapper falls back to box rasterization.
- setup.sh has stale references to requirements.txt and run_app.sh.

15. Model References

Model	Purpose	Reference
Grounding DINO	Zero-shot object detection	arxiv.org/abs/2303.05499

Depth Anything V2	Relative monocular depth	github.com/DepthAnything/Depth-Anything-V2
SAM2	Optional segmentation refinement	github.com/facebookresearch/sam2
LFM2.5-VL-1.6B	On-device vision-language model	liquid.ai
Cactus Engine	On-device inference runtime (ANE)	github.com/cactus-compute/cactus
Gemini 2.5 Flash Lite	Cloud fallback + DINO prompt extraction	cloud.google.com/vertex-ai
Gemini 2.5 Pro	RCP telemetry extraction	cloud.google.com/vertex-ai

16. Future Extensions

- RTSP / live camera source. The pipeline reads from cv2.VideoCapture, which already accepts RTSP URLs.
- Temporal object tracking. Assigning persistent IDs across frames would reduce flicker and enable trajectory-based prediction.
- Dynamic corridor adaptation. Ego-motion estimation or SLAM could reshape the corridor in real time.
- Depth-weighted hazard scoring. NEAR obstacles should contribute more to occupancy than FAR ones.
- Raspberry Pi 5 deployment. Classical CV + lightweight VLM on a Pi 5 with camera module.
- Haptic/wearable alerting. Vibration patterns on a smartwatch for noisy environments.
- Live streaming integration. WebSocket bridge between the two pages for real-time prompt updates.
- Performance evaluation based on generated prompts. Measure detection recall and precision of dynamic vs. static prompts.

Appendix A: Key Code Excerpts

A.1 Classical CV Fallback (pathguard/fallback.py)

```
from dataclasses import dataclass
from typing import Dict

import cv2
import numpy as np


@dataclass
class FallbackParams:
    canny_low: int = 60
    canny_high: int = 160
    min_component_area: int = 250
    trip_component_area: int = 180
    trip_aspect_ratio: float = 6.0
    blur_kernel: int = 5
    morph_kernel: int = 5


def detect_fallback_obstacles(
    frame_bgr: np.ndarray,
    corridor_mask: np.ndarray,
    blur_thresh: float,
    params: FallbackParams | None = None,
) -> Dict[str, object]:
    cfg = params or FallbackParams()

    gray = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2GRAY)
```

```
    blur = cv2.GaussianBlur(gray, (cfg.blur_kernel, cfg.blur_kernel), 0)

    lap_var = float(cv2.Laplacian(blur, cv2.CV_64F).var())
    low_quality = lap_var < blur_thresh

    edges = cv2.Canny(blur, cfg.canny_low, cfg.canny_high)
    edges = cv2.bitwise_and(edges, corridor_mask)

    kernel = np.ones((cfg.morph_kernel, cfg.morph_kernel), dtype=np.uint8)
    merged = cv2.morphologyEx(edges, cv2.MORPH_CLOSE, kernel, iterations=1)
    merged = cv2.dilate(merged, kernel, iterations=1)

    num_labels, labels, stats, _ = cv2.connectedComponentsWithStats((merged > 0).astype(np.uint8), connectivity=8)

    obstacle_mask = np.zeros_like(corridor_mask, dtype=np.uint8)
    trip_risk_flag = False

    for comp_id in range(1, num_labels):
        x, y, w, h, area = stats[comp_id]
        if area < cfg.min_component_area:
            continue
        obstacle_mask[labels == comp_id] = 255

        if area >= cfg.trip_component_area:
            ratio = max((w / max(1.0, h)), (h / max(1.0, w)))
            if ratio >= cfg.trip_aspect_ratio:
                trip_risk_flag = True

    obstacle_mask = cv2.bitwise_and(obstacle_mask, corridor_mask)
    corridor_area = max(1, int(np.count_nonzero(corridor_mask)))
    obstacle_area = int(np.count_nonzero(obstacle_mask))
    occupancy_score = float(obstacle_area / corridor_area)

    return {
        "obstacle_mask": obstacle_mask,
        "occupancy_score": occupancy_score,
        "trip_risk_flag": bool(trip_risk_flag),
        "low_quality": bool(low_quality),
        "laplacian_var": lap_var,
    }
```

A.2 Event State Machine (pathguard/events.py)

```
from dataclasses import dataclass
from typing import Dict, List, Optional


STATE_CLEAR = "CLEAR"
STATE_OBSTACLE = "OBSTACLE AHEAD"
STATE_TRIP = "TRIP RISK"


@dataclass
class EventRecord:
    timestamp_s: float
    state: str
    occupancy_score: float

    def as_dict(self) -> Dict[str, object]:
        return {
            "timestamp_s": round(self.timestamp_s, 2),
            "state": self.state,
            "occupancy_score": round(self.occupancy_score, 4),
```

```
        }


class EventStateMachine:
    def __init__(self, persistence_frames: int = 4, debounce_sec: float = 2.0, max_events: int = 200) -> None:
        self.persistence_frames = persistence_frames
        self.debounce_sec = debounce_sec
        self.max_events = max_events

        self.current_state = STATE_CLEAR
        self._hazard_counter = 0
        self._last_event_ts: Dict[str, float] = {}
        self.events: List[EventRecord] = []

    def _append_event(
        self,
        timestamp_s: float,
        state: str,
        occupancy_score: float,
    ) -> None:
        last_ts = self._last_event_ts.get(state, -1e9)
        if timestamp_s - last_ts < self.debounce_sec:
            return
        self._last_event_ts[state] = timestamp_s

        self.events.append(
            EventRecord(
                timestamp_s=timestamp_s,
                state=state,
                occupancy_score=occupancy_score,
            )
        )
        if len(self.events) > self.max_events:
            self.events = self.events[-self.max_events :]

    def update(
        self,
        timestamp_s: float,
        occupancy_score: float,
        occ_thresh: float,
        trip_risk_flag: bool,
        low_quality: bool = False,
    ) -> str:
        hazard_now = (not low_quality) and (occupancy_score > occ_thresh)
        if hazard_now:
            self._hazard_counter += 1
        else:
            self._hazard_counter = 0

        target_state = STATE_CLEAR
        if self._hazard_counter >= self.persistence_frames:
            target_state = STATE_TRIP if trip_risk_flag else STATE_OBSTACLE

        if target_state != self.current_state:
            self.current_state = target_state
            self._append_event(timestamp_s, target_state, occupancy_score)

        return self.current_state

    def events_as_rows(self) -> List[Dict[str, object]]:
        return [event.as_dict() for event in self.events]
```

A.3 Hybrid Cloud Routing (narrator/cactus_vl.py, lines 162-226)

```
def describe_image(
        self,
        image_path: str,
        prompt: str,
        system: str = DEFAULT_SYSTEM,
        max_tokens: int = 96,
        temperature: float = 0.1,
        top_p: float = 0.95,
        confidence_threshold: float = 0.90,
        cooldown_seconds: float = 10.0,
    ) -> Dict[str, Any]:
        self.load()

        img = str(Path(image_path).resolve())
        messages = [{
            "role": "user",
            "content": system + "\n\n" + prompt,
            "images": [img]
        }]

        with self._lock:
            raw = cactus_complete(
                self._model,
                messages,
                max_tokens=max_tokens,
                temperature=temperature,
                top_p=top_p,
            )

        try:
            local_out = json.loads(raw)
            local_success = local_out.get("success", True)  # Some old bindings don't explicitly return 'success'
            local_conf = local_out.get("confidence", 1.0)

            # --- TRUE HYBRID ROUTING LOGIC ---
            if local_success and local_conf >= confidence_threshold:
                local_out["cloud_handoff"] = False
                return local_out

            # If we reach here, local model failed or confidence is too low.
            now_ms = time.time() * 1000.0
            time_since_last_call = (now_ms - self._last_gemini_call_ms) / 1000.0

            if time_since_last_call < cooldown_seconds:
                return {
                    "success": True,
                    "response": f"[Local model uncertain (Confidence {local_conf:.2f}). Cloud API on cooldown for {in
                    "cloud_handoff": False,
                    "confidence": local_conf,
                    "total_time_ms": local_out.get("total_time_ms", 0),
                    "raw": local_out
                }

            # Off cooldown! Hit the Vertex API
            self._last_gemini_call_ms = now_ms
            cloud_out = self._call_gemini_fallback(img, prompt)

            # Merge local metrics for visibility if cloud succeeds
            if cloud_out.get("success"):
                 cloud_out["total_time_ms"] += local_out.get("total_time_ms", 0)

            return cloud_out

        except Exception as e:
```

```
            return {"success": False, "error": f"Serialization error: {str(e)}", "raw": raw}
```

A.4 Dynamic Prompt Merging (integration/dynamic_prompts.py)

```
"""
Dynamic Prompt Bridge: Cactus Narrator -> PathGuard

Reads the dot-separated noun list generated by Cactus's generate_dino_prompt()
and converts it into a clean Python list for PathGuard's GroundedDINO detector.

Workflow:
    1. Cactus Narrator processes video -> generates transcript
    2. generate_dino_prompt() extracts nouns -> saves to *_dino_prompt.txt
    3. This module reads that file -> returns List[str] for PathGuard's RuntimeParams.prompts

Example:
    Input text:  "crane . worker . tractor . safety vest . rebar ."
    Output list: ["crane", "worker", "tractor", "safety vest", "rebar"]
"""

from pathlib import Path
from typing import List, Optional


def parse_dino_prompt_text(text: str) -> List[str]:
    """Parse a dot-separated Cactus DINO prompt string into a list of noun phrases.

    Args:
        text: Dot-separated string like "crane . worker . tractor ."

    Returns:
        Deduplicated list of lowercase noun phrases.
    """
    if not text or not text.strip():
        return []

    # Split on dots, strip whitespace, drop empties
    raw = [tok.strip().lower() for tok in text.split(".")]
    nouns = [tok for tok in raw if tok]

    # Deduplicate while preserving order
    seen = set()
    result = []
    for noun in nouns:
        if noun not in seen:
            seen.add(noun)
            result.append(noun)
    return result


def load_dino_prompt_file(filepath: str) -> List[str]:
    """Read a Cactus *_dino_prompt.txt file and parse its contents.

    Args:
        filepath: Path to a dino_prompt.txt file generated by Cactus Narrator.

    Returns:
        List of noun phrases, or empty list if file doesn't exist / is empty.
    """
    path = Path(filepath)
    if not path.exists() or not path.is_file():
        return []
```

```
    text = path.read_text(encoding="utf-8").strip()
    return parse_dino_prompt_text(text)


def merge_prompts(
    static: List[str],
    dynamic: List[str],
    max_prompts: int = 50,
) -> List[str]:
    """Merge static (default) and dynamic (Cactus-generated) prompt lists.

    Dynamic prompts are prepended (higher priority for scene-specific labels),
    followed by static prompts not already in the dynamic set.
    Capped at max_prompts to avoid GroundedDINO tokenizer overflow.

    Args:
        static:  Default PathGuard prompts from config.PROMPTS.
        dynamic: Cactus-generated prompts from a DINO prompt file.
        max_prompts: Maximum number of prompts (default 50).

    Returns:
        Merged, deduplicated list with dynamic prompts first, capped.
    """
    if not dynamic:
        return list(static)[:max_prompts]

    seen = set()
    result = []

    # Dynamic first (scene-specific, higher priority)
    for prompt in dynamic:
        if len(result) >= max_prompts:
            break
        key = prompt.lower().strip()
        if key and key not in seen:
            seen.add(key)
            result.append(prompt)

    # Static second (fill in anything the dynamic list missed)
    for prompt in static:
        if len(result) >= max_prompts:
            break
        key = prompt.lower().strip()
        if key and key not in seen:
            seen.add(key)
            result.append(prompt)

    return result
```

Closing Note

PathGuard was built for the UMD x Ironsite Startup Shell Hackathon 2026. The core conviction is that spatial safety on construction sites should not depend on any single model being available, fast, or correct. Every layer in this system -- from Canny edges to Gemini 2.5 Pro -- is optional. The system gets better as you add layers, but it never goes dark when you remove them.